AI Shield: My Capstone Journey

A simple reference guide | Abdelrahman Soliman

The project in one paragraph

AI Shield is my capstone project about detecting AI-generated images and showing useful evidence about an image's origin. It began with a face-manipulation detector, expanded into a system combining several models, and then moved toward a stronger single detector after more demanding tests. The current version combines fine-tuned Community Forensics with metadata and signed-provenance reporting in a working local backend.

What I am trying to answer

- **Detection:** Can a model distinguish real and generated images, including images from generators excluded from my fine-tuning?
- **Evidence:** What information about creation, editing, or signing is actually recorded in the uploaded file?
- **Reliability:** Can I reproduce the research results when I move the model into an application?

These are related questions, but they are not the same. A detection score is a prediction. Metadata is a recorded claim. A signed credential can help verify that a claim belongs to a file. None of these automatically proves that a depicted event really happened.

How to use this guide

Pages 2-3 explain the first models and original combined system. Pages 4-6 explain the harder evaluation, fine-tuning, and results. Page 7 explains the frozen detector. Pages 8-9 explain provenance and the backend. Page 10 lists the current status, next steps, and important links.

The story follows development stages rather than exact dates. When different experiments use different datasets, their results are kept separate.

What I should remember: The main achievement is not just a high score. It is the process of building models, discovering their limits, testing alternatives fairly, and turning the chosen model into a reproducible evidence-reporting system.

2. The First Models: From Faces to General Images

First working prototype: face manipulation

My first trained detector used EfficientNetV2-B0 with OpenForensics, a dataset focused on manipulated faces. EfficientNet is an image-classification architecture: it learns visual patterns and uses them to make a prediction. I also built an image-upload demo so the model could be tried outside the training notebook.

Validation accuracy was 96.51%, but accuracy on the held-out test was 85.77%. Validation data helps guide development; held-out test data is reserved for evaluating the selected model. The lower test result showed why I could not judge reliability from validation alone.

This model was also too narrow for the final goal. Many generated images contain no faces, so a face-manipulation detector is not a general AI-image detector.

Expanding the forensic layer

I then worked with EfficientNet-B0 on Tiny-GenImage to learn broader visual artifacts. This stage used a balanced 28,000-image training subset and 7,000 original validation images. I calibrated classifier scores, meaning I fitted an adjustment to improve their interpretation, and cached predictions so later comparisons could reuse the same outputs.

I also built a real-only variational autoencoder, or VAE. Its encoder compresses an image and its decoder reconstructs it. The idea was that a model trained on real images might reconstruct generated images less successfully. I implemented the reconstruction-based training approach rather than assuming it would work.

The VAE's standalone AUROC was about 0.470, which was poor. A simple weighted combination also hurt performance. This was a useful negative result: a plausible idea did not earn a place merely because it sounded complementary.

FaceGate made the face branch conditional: use it only when a suitable face is present. No face means that branch is inapplicable, not that the image is real.

What I should remember: Different detectors answer different questions. I had to test each signal and handle when it did not apply.

3. The Semantic Layer and Original Fusion

A custom attention head

I explored whether relationships between image regions could add information beyond local artifacts. I used a pretrained Vision Transformer, or ViT, which represents an image as small patches. On top of that backbone, I built a custom head with multi-head attention, refinement, and attention pooling to produce an image-level score.

The backbone stayed frozen while 856,578 parameters in the custom head were trained. This is an important distinction: I reused a pretrained visual representation and designed and trained an additional component. I did not train the whole transformer from scratch.

I called this the semantic layer, but it was visual modeling, not a proven system for checking captions, factual claims, or human-like reasoning. Attention can learn relationships; it does not automatically explain an image's truth.

Learning how to combine the signals

I aligned forensic and semantic outputs for the same images, then trained a GLM: here, logistic regression, a small model that learns how to combine input features. The original full fusion used 17 features: 11 forensic and 6 semantic.

The original Tiny-GenImage fusion split contained 4,200 training images, 1,400 validation images, and 1,400 test images. The full system reached AUROC 0.9819, accuracy 94.07%, and F1 0.9406 on that 1,400-image test. It classified 1,317 images correctly.

Compared with the calibrated EfficientNet baseline on that evaluation, it caught 54 additional generated images while adding one false positive. This supported using fusion for that particular benchmark.

However, those numbers did not establish that the system would generalize to every generator or image source. Later results exposed that limitation.

What I should remember: The original fusion was a real improvement on its own test set. It is different from the later CF fusion experiment and is not the current selected detector.

4. The Turning Point: A Harder Benchmark

Checking whether the old success carried over

I evaluated the saved original AI Shield pipeline without retraining it on a newer collection. This was a consistency and generalization audit: checking both that the pipeline behaved as intended and that its earlier success transferred to different images.

The old full fusion reached only about 0.606 AUROC on the ordinary new test and 0.599 on the separate GPT Image 2 evaluation. These results were much weaker than its original benchmark result.

This did not erase the earlier experiment. It showed that good performance on one dataset was not enough. Dependence on older image sources or generator-specific patterns is a possible explanation, not a proven diagnosis of every error.

Building Dataset v1

Dataset v1 contained 5,124 images: 2,562 real and 2,562 generated. I recorded image identities, split membership, and file hashes so experiments could use consistent examples.

- **Training:** 2,990 images, used to update the model's parameters.
- **Validation:** 640 images, used to choose checkpoints and thresholds.
- **Ordinary test:** 640 images, used for final evaluation after selection.
- **Held-out-generator evaluation:** 854 images, kept separate to test transfer.

Development used five generator families: FLUX.2 max, GLM-Image, Seedream 5, Nano Banana 2, and Qwen Image 2. GPT Image 2 was excluded from this project's fine-tuning and validation, then used in the separate held-out evaluation.

Here, unseen means excluded from my project's fine-tuning and selection. It does not prove that the original pretrained backbone never encountered related content.

I saved per-image predictions and disagreement tables so I could inspect failures, not only average scores.

What I should remember: Fair model comparison means running models on the same final images. Results from different datasets must not be presented as one continuous improvement curve.

5. Fine-Tuning Community Forensics

Why I changed the main detector

Community Forensics, abbreviated CF, offered a stronger starting point for general AI-image detection. I first measured its pretrained performance, then adapted it using Dataset v1. Fine-tuning means continuing training from existing learned weights rather than starting from random values.

The starting model was OwensLab/commfor-model-384. I pinned its revision, meaning I recorded the exact published version rather than relying on a name that might later point to different files.

Two training stages

Stage 1 trained only the classifier head for three epochs. The head is the final component that turns visual features into the detection output. Most of the pretrained model remained unchanged.

Stage 2 trained the final two transformer blocks, final normalization, and classifier head for another three epochs. This allowed some higher-level features to adapt while avoiding a full-model retraining exercise.

Validation AUROC increased from 0.8780 for the pretrained baseline to 0.9804 after Stage 1 and 0.9870 after Stage 2. I selected the checkpoint using validation performance, rather than choosing it based on the final test.

I also dealt with practical issues, including handling the first frame of MPO-style JPEG files and replacing a corrupted cached pretrained checkpoint with a verified copy. These details matter because an experiment is only useful if it actually loads and processes the intended data and weights.

What was my contribution?

I did not invent Community Forensics. My contribution was preparing the evaluation setup, adapting the model in controlled stages, comparing it with the existing system on the same images, checking errors, and making the selected version reproducible.

What I should remember: Training improved the measured results, but choosing the right data split, comparison, and saved checkpoint was just as important as running the training loop.

6. What the Results Actually Say

Same images, three detectors

Ordinary-test AUROC: old AI Shield full fusion about 0.606; original CF 0.8860; fine-tuned CF 0.9884.

GPT Image 2 evaluation AUROC: old full fusion about 0.599; original CF 0.9555; fine-tuned CF 0.9946.

Fine-tuned CF accuracy was 95.16% on the ordinary test and 96.60% on the held-out-generator evaluation. On the ordinary test, the real-image false-positive rate fell from 20.63% for original CF to 3.75% for fine-tuned CF, using each model's validation-selected max-F1 threshold.

The metrics in simple terms

- **Accuracy:** the share of all images classified correctly.
- **Precision:** among images called generated, how many really were generated?
- **Recall:** among generated images, how many did the detector catch?
- **F1:** one number balancing precision and recall; it becomes low when either is low.
- **False-positive rate:** the share of real images incorrectly flagged as generated.
- **AUROC:** how well scores rank generated images above real images across thresholds. About 0.5 is chance-level ranking; 1.0 is perfect. It is not percentage accuracy.

Why I did not keep adding models

I tried a new fusion combining fine-tuned CF, calibrated EfficientNet, and FaceGate/OpenForensics through a new logistic-regression model. It reached AUROC 0.9854 on the ordinary test and 0.9936 on the held-out evaluation, slightly below CF alone on both.

I therefore selected CF alone. This was a different fusion from the original 17-feature system. The VAE was not included.

These are Dataset v1 results, not universal guarantees. Improvement was not uniform: Qwen Image 2 recall declined slightly, from about 98.4% to 96.9%. JPEG/resize robustness and broader content categories still need testing.

What I should remember: The simpler detector won the measured comparison. More models did not automatically mean better detection.

7. Freezing and Verifying the Selected Detector

What freezing means here

I stopped changing the selected detector so I could build the application around a stable version. The frozen unit includes the fine-tuned best.pt weights, model architecture, preprocessing, label mapping, and chosen threshold. It does not mean the model can never improve.

The default threshold is 0.6339887976646423, approximately 0.634. A score at or above that value is classified as generated. This threshold was selected for the highest validation F1, not guessed from the test set. The saved alternative, 0.7482541799545288, is a lower-false-positive operating point, but is not the application default.

The exact image preparation

Use the first frame, apply EXIF orientation, convert to RGB, resize the shorter side to 440 pixels with bilinear antialiasing, center-crop to 384 x 384, and apply ImageNet normalization. Labels are 0 = real and 1 = generated. Changing this preparation can change predictions even when the weights are identical.

Why record a hash?

A SHA-256 hash is a digital fingerprint of a file's bytes. Recording it prevents accidentally using another file also named best.pt. The selected fine-tuned weights have a hash beginning 51532d28. The pretrained weights and the unrelated repository-root best.pt are different files.

I exported a standalone inference function and checked six fixed validation images against their saved predictions. Their classifications matched, and score differences stayed within the recorded numerical tolerances. The same images also passed checks through the local HTTP API.

These checks prove implementation parity: the application reproduces the saved calculation. Six reference images do not establish general detection accuracy; the larger evaluation serves that purpose.

The API calls the output an AI score. Applying sigmoid produces a bounded score but does not, by itself, prove that a score of 0.94 means a calibrated 94% probability.

What I should remember: Freezing makes the trained model a dependable component instead of a moving target.

8. The Provenance Layer: Evidence, Not Another Detector

What this layer does

The provenance layer reads information already recorded in the original file. It is software using ExifTool and a C2PA verification SDK, not a newly trained AI model. It needs test cases, but it does not need a labeled training dataset just to extract tags and verify credentials.

EXIF is a common metadata format. Depending on the file, metadata can include camera/device details, editing software, creation or modification dates, and GPS. ExifTool reads supported metadata structures and returns structured data; an AI model is not asked to interpret an empty JSON object.

Embedded dates are kept separate from filesystem dates, which can change when a file is copied. Conflicting metadata values remain visible. GPS extraction is optional and off by default because location is sensitive.

What C2PA adds

C2PA is a standard for signed content credentials: records that can describe creation, editing, and ingredients. My code checks three separate things: the signature, whether the claims are bound to the current image, and whether the signer is trusted under the configured policy.

Credentials can be absent, valid but untrusted, failed, incomplete, unsupported, or unavailable. The verifier runs offline: it does not retrieve remotely referenced manifests or obtain live revocation responses. Those limits must remain visible.

Rules I must not forget

- Missing metadata means unknown, not fake. Websites and ordinary editing can remove metadata.
- Photoshop in the metadata means recorded editing software, not proof of AI generation.
- Camera tags are unverified claims unless supported by appropriate evidence; they are not proof of authenticity.
- A trusted signed record does not prove that a photographed scene really happened.

The provenance evidence does not change CF's score. It sits next to detection in the report. Predicting missing source information from pixels would be a separate research problem; SDAIE was discussed but was not added.

What I should remember: This layer reports what is available and what can be verified. It must not invent an origin when evidence is missing.

9. Connecting Both Branches in a Local Backend

The combined flow

Original image upload -> preserve the file -> extract metadata and verify credentials -> run frozen CF on a separately decoded/prepared image -> return one report containing detection and evidence.

A backend is the service behind a website. I built this one with FastAPI, a Python framework for receiving requests and returning structured results. The API is the connection the future website will use; the detector does not need to live inside the frontend.

The /analyze endpoint accepts an image upload. The /health endpoint reports readiness and detector identity. When the local server is running, http://127.0.0.1:8000/docs provides an interactive way to try the API. Localhost means this computer, not a publicly accessible website.

Practical handling already implemented

The service accepts decoded JPEG, PNG, and WebP images. It limits uploads to 25 MiB and images to 20 million pixels, checks image validity, limits concurrent analysis, and deletes temporary uploads after processing. The original file stays intact for evidence extraction.

If metadata extraction fails, the detector can still return a result with an evidence error. If detection fails, the service returns an error rather than making up a score. The public-facing response leaves out raw metadata/manifests and internal error details, although the local inspection tool can provide more detail.

What was tested

The recorded verification run passed 38 automated tests covering provenance and backend behavior. Provenance cases included a camera photo, an edited copy, metadata-free content, signed credentials, untrusted credentials, and tampered signed pixels. Six saved detector images also matched expected predictions through actual HTTP requests.

These checks establish software behavior, not higher classification accuracy. Adding metadata has not yet demonstrated an improvement to CF's detection metrics because it does not alter the detector's output.

What I should remember: The backend connection works locally. Public deployment still needs server-level processing limits, timeouts, HTTPS, and validation on the chosen host.

10. Current Status, Next Steps, and Useful Links

Where the project stands

Working locally: the selected CF detector, verified inference, EXIF/C2PA reporting, and a combined upload API. The provenance code has a local Git commit; the backend is not yet committed/pushed. The standalone website and public backend are not deployed. The existing Hugging Face demo still runs the older fusion system, not this final CF pipeline.

My next steps, in order

- Run JPEG compression and resizing tests using fixed images and settings. Measure changes in scores, recall, and false positives.
- Inspect failures across human artwork, designs, conventional 3D renders, and edited photographs. Not being a camera photo does not make an image AI-generated.
- Commit and publish the intended backend code, documenting the weights and configuration without adding private uploads or large artifacts to Git.
- Build the upload/results website. Show the AI score, metadata, and credential checks separately, with clear unknown/error states.
- Deploy and validate the complete pipeline on the chosen server. Vercel/Oracle was a proposed arrangement, not a completed deployment.
- Consider semantic/claim consistency and short-video support later, after validating the image workflow.

Terms I am likely to forget

Checkpoint = saved model weights. Inference = using a model to predict. Threshold = score cutoff. Fusion = combining model signals. Hash = file fingerprint. Provenance = recorded origin/edit history. API = a defined way for programs to communicate.

Open these when I need the details

Project overview and progress

CF training notebook

Same-image detector comparison

New fusion experiment

Frozen detector verification and exports

GitHub repository (published state may be older)